

GCP Deployment Guide — AskMyNotes

Complete Step-by-Step for First-Time Deployers

Before You Start — What You Need

- ☐ A Google account (Gmail works)
- ☐ Your AskMyNotes project code on your local machine
- ☐ A GitHub account with your repo pushed
- ☐ Your API keys ready: QDRANT_URL, QDRANT_API_KEY, GROQ_API_KEY, LANGCHAIN_API_KEY

Time needed: ~45 minutes first time. 3 minutes every deploy after that.

PART 1 — GCP Account Setup (One Time)

Step 1.1 — Create GCP Account

1. Go to <https://cloud.google.com>
2. Click "**Get started for free**"
3. Sign in with your Google account
4. GCP will ask for a credit/debit card — **this is required** even for free tier
 - You will NOT be charged unless you manually upgrade
 - Free tier accounts have a hard limit — no surprise bills
 - GCP gives \$300 free credits for 90 days on top of the always-free tier
5. Select your country: **India**
6. Accept terms and click "**Start my free trial**"

⚠ **If you don't have a credit card:** Use a virtual card from your bank app (most Indian banks — HDFC, ICICI, SBI — provide virtual debit cards in their app). Prepaid cards with ₹0 balance also work for verification.

Step 1.2 — Install gcloud CLI

The gcloud CLI is the command-line tool that lets you control GCP from your terminal.

On macOS:

```
bash
```

```
# Option A – Homebrew (recommended if you have Homebrew)
brew install google-cloud-sdk

# Option B – Manual install
curl https://sdk.cloud.google.com | bash
exec -l $SHELL # Restart your shell
```

On Ubuntu/Linux:

```
bash

sudo apt-get install apt-transport-https ca-certificates gnupg curl
curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/cloud.google.gpg
echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] \
  https://packages.cloud.google.com/apt cloud-sdk main" | \
  sudo tee -a /etc/apt/sources.list.d/google-cloud-sdk.list
sudo apt-get update && sudo apt-get install google-cloud-cli
```

On Windows:

1. Download the installer from: <https://cloud.google.com/sdk/docs/install#windows>
2. Run the `.exe` installer
3. Check "Install bundled Python" during setup
4. Open a new Command Prompt after install (not PowerShell)

Verify install worked:

```
bash

gcloud version
# Should print: Google Cloud SDK 460.x.x
```

Step 1.3 – Login to gcloud

```
bash

gcloud auth login
```

This opens a browser window. Sign in with the same Google account you used to create GCP.

What you see in terminal after success:

```
You are now logged in as [your-email@gmail.com]
```

PART 2 — Create GCP Project (One Time)

A "project" in GCP is a container for all your resources — like a folder for your app.

Step 2.1 — Create the Project

```
bash
```

```
gcloud projects create askmynotes-project --name="AskMyNotes"
```

Common error: `Project ID already exists`

- Project IDs are globally unique across all GCP users
- Fix: add your name or random suffix: `askmynotes-satyam-2025`
- Use the same ID everywhere in this guide

Verify it was created:

```
bash
```

```
gcloud projects list
```

```
# Shows a table — askmynotes-project should appear
```

Step 2.2 — Set This Project as Default

```
bash
```

```
gcloud config set project askmynotes-project
```

Every gcloud command after this will use `askmynotes-project` automatically.

Verify:

```
bash
```

```
gcloud config get project
```

```
# Prints: askmynotes-project
```

Step 2.3 — Link Billing Account to Project

GCP requires a billing account linked to use Cloud Run (even on free tier).

1. Go to <https://console.cloud.google.com/billing>
2. Click your billing account name
3. Click "**Manage linked projects**" in the left panel
4. Click "**Link a project**"

5. Select `askmynotes-project`

6. Click **Save**

Alternative via CLI:

```
bash

# First get your billing account ID
gcloud billing accounts list
# Output example: XXXXXX-YYYYYY-ZZZZZZ

# Link it
gcloud billing projects link askmynotes-project \
  --billing-account=XXXXXX-YYYYYY-ZZZZZZ
```

PART 3 — Enable APIs (One Time)

GCP services are disabled by default. You must enable what you need.

```
bash

gcloud services enable \
  run.googleapis.com \
  artifactregistry.googleapis.com \
  cloudbuild.googleapis.com
```

What each does:

- `run.googleapis.com` — Cloud Run (where your app lives)
- `artifactregistry.googleapis.com` — Docker image storage
- `cloudbuild.googleapis.com` — Builds your Docker image on GCP servers

This takes 1-2 minutes. You'll see a progress spinner. Wait for it to complete.

Verify APIs are enabled:

```
bash

gcloud services list --enabled | grep -E "run|artifact|build"
# Should show all 3 APIs as ENABLED
```

PART 4 — Create Artifact Registry Repository (One Time)

Artifact Registry is where your Docker images are stored. Think of it as a private Docker Hub for your

GCP project.

```
bash

gcloud artifacts repositories create askmynotes-repo \
  --repository-format=docker \
  --location=us-central1 \
  --description="AskMyNotes Docker images"
```

Why us-central1? It has the most generous free tier quotas. Always use the same region throughout this guide.

Verify:

```
bash

gcloud artifacts repositories list
# Should show askmynotes-repo in us-central1
```

Step 4.1 — Configure Docker to Use GCP Registry

This lets Docker on your machine push images to GCP:

```
bash

gcloud auth configure-docker us-central1-docker.pkg.dev
```

Output you should see:

```
Adding credentials for: us-central1-docker.pkg.dev
Docker configuration file updated.
```

PART 5 — Prepare Your App for Deployment

Step 5.1 — Check Your Dockerfile Exists

Your project root should have a `Dockerfile`. If not, create it:

```
dockerfile
```

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Remove .env if accidentally copied – secrets come from Cloud Run env vars
RUN rm -f .env

EXPOSE 8080

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Important: Port must be **8080** for Cloud Run — not 8000.

Step 5.2 — Create `.dockerignore`

Create `.dockerignore` in your project root:

```
.env
.env.local
.git
.gitignore
__pycache__
*.pyc
*.pyo
.pytest_cache
.mypy_cache
*.egg-info
dist/
build/
node_modules/
*.md
.DS_Store
mlruns/
eval/eval_results.csv
data/raw/
```

This prevents secrets and junk from being included in your image.

Step 5.3 — Verify Your App Runs Locally First

```
bash
```

```
# Test with Docker locally before pushing to GCP
```

```
docker build -t askmynotes-local .
```

```
docker run -p 8080:8080 \
  -e QDRANT_URL="your-qdrant-url" \
  -e QDRANT_API_KEY="your-key" \
  -e GROQ_API_KEY="your-key" \
  -e LANGCHAIN_API_KEY="your-key" \
  -e LANGCHAIN_TRACING_V2="true" \
  -e LANGCHAIN_PROJECT="askmynotes" \
  askmynotes-local
```

```
# In a new terminal:
```

```
curl http://localhost:8080/health
```

```
# Should return: {"status": "ok", ...}
```

If this works, your app is ready. If not, fix it before going further.

PART 6 — Build & Push Docker Image to GCP

Option A — Build on GCP Servers (Recommended, Easier)

Cloud Build builds the image on GCP's servers — you don't need Docker running locally with lots of RAM.

```
bash
```

```
# Run this from your project root (where Dockerfile is)
```

```
cd /path/to/your/askmynotes/project
```

```
gcloud builds submit \
  --tag us-central1-docker.pkg.dev/askmynotes-project/askmynotes-repo/askmynotes \
  --timeout=15m
```

What happens:

1. gcloud zips your project folder and uploads it to GCP
2. GCP builds the Docker image on their servers
3. The built image is stored in Artifact Registry

You'll see a stream of build logs in your terminal. This takes 3-8 minutes first time.

Successful output ends with:

```
SUCCESS
```

```
ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

```
...
```

```
Images: us-central1-docker.pkg.dev/askmynotes-project/askmynotes-repo/askmynotes
```

Common error: `INVALID_ARGUMENT: Build failed: ...`

- Check your Dockerfile has no syntax errors
- Make sure requirements.txt exists in the same folder as Dockerfile

Option B — Build Locally, Push to GCP

Use this if Cloud Build fails or you want faster iterations:

```
bash
```

```
# Build locally
```

```
docker build -t us-central1-docker.pkg.dev/askmynotes-project/askmynotes-repo/askmynotes
```

```
# Push to GCP registry
```

```
docker push us-central1-docker.pkg.dev/askmynotes-project/askmynotes-repo/askmynotes
```

Verify image is in registry:

```
bash
```

```
gcloud artifacts docker images list \
```

```
  us-central1-docker.pkg.dev/askmynotes-project/askmynotes-repo
```

```
# Should show your image with a recent timestamp
```

PART 7 — Deploy to Cloud Run

Step 7.1 — First Deployment

```
bash
```

```
gcloud run deploy askmynotes \
  --image us-central1-docker.pkg.dev/askmynotes-project/askmynotes-repo/askmynotes \
  --platform managed \
  --region us-central1 \
  --allow-unauthenticated \
  --memory 512Mi \
  --cpu 1 \
  --min-instances 0 \
  --max-instances 2 \
  --port 8080 \
  --timeout 120
```

Flags explained:

Flag	What it does	Why
<code>--allow-unauthenticated</code>	Public URL, no login needed	So you can share the demo
<code>--memory 512Mi</code>	Max RAM per instance	Free tier limit
<code>--cpu 1</code>	1 vCPU	Free tier limit
<code>--min-instances 0</code>	Scale to zero	Critical for free tier — no idle charges
<code>--max-instances 2</code>	Cap instances	Prevents runaway costs
<code>--port 8080</code>	Container port	Must match your Dockerfile EXPOSE
<code>--timeout 120</code>	Request timeout 2 min	RAG queries can take ~30s

After running, you'll see:

```
Deploying container to Cloud Run service [askmynotes] in project [askmynotes-project] region [us-central1]
✓ Deploying... Done.
✓ Routing traffic...
✓ Setting IAM Policy...
Done.
Service [askmynotes] revision [askmynotes-00001-xxx] has been deployed and is serving 100 percent of traffic.
Service URL: https://askmynotes-xxxxxxxxxx-uc.a.run.app
```

Save that URL — it's your live app.

Step 7.2 — Set Environment Variables (Secrets)

Never put API keys in your Docker image. Set them separately in Cloud Run:

```
bash

gcloud run services update askmynotes \
  --region us-central1 \
  --set-env-vars "\
QDRANT_URL=https://your-cluster.us-east4-0.gcp.cloud.qdrant.io:6333,\
QDRANT_API_KEY=your-qdrant-api-key-here,\
GROQ_API_KEY=your-groq-api-key-here,\
LANGCHAIN_API_KEY=lsv2_your-langsmith-key,\
LANGCHAIN_TRACING_V2=true,\
LANGCHAIN_PROJECT=askmynotes"
```

Important syntax notes:

- No spaces around the `=`
- No spaces after commas between variables
- Wrap the entire value in double quotes
- If any value contains a comma, wrap that specific value in single quotes

Step 7.3 — Test Your Live App

```
bash

# Get your service URL
gcloud run services describe askmynotes --region us-central1 \
  --format="value(status.url)"

# Test health endpoint
curl https://askmynotes-xxxxxxxxxx-uc.a.run.app/health
# Should return: {"status": "ok", "qdrant_connected": true}
```

If health check fails:

```
bash

# Check logs
gcloud logging read "resource.type=cloud_run_revision \
  AND resource.labels.service_name=askmynotes" \
  --limit 50 \
  --format "table(timestamp, textPayload)"
```

PART 8 — Set Up Billing Alert (\$0 Protection)

Do this before anything else — takes 2 minutes.

1. Go to <https://console.cloud.google.com/billing>
2. Click your billing account name (format: `My Billing Account`)
3. In the left sidebar click "**Budgets & alerts**"
4. Click "+ **CREATE BUDGET**"
5. Fill in:
 - **Name:** `Zero spend alert`
 - **Projects:** Select `askmynotes-project`
 - **Services:** All services
 - **Amount:** `$0.01` (GCP minimum — effectively \$0)
6. Under "**Alert thresholds**" add:
 - 50% of budget (\$0.005)
 - 100% of budget (\$0.01)
7. Under "**Manage notifications**":
 - Check "Email alerts to billing admins and users"
 - Add your email address
8. Click "**Finish**"

You will now get an email the moment anything is charged. On the free tier this should never trigger.

PART 9 — GitHub Actions CI/CD (Auto-Deploy on Push)

Every time you push to `main`, GitHub will automatically build and deploy your app. No manual steps.

Step 9.1 — Create Service Account for GitHub Actions

```
bash

# Create a dedicated service account for GitHub
gcloud iam service-accounts create github-actions-sa \
  --display-name="GitHub Actions SA" \
  --project=askmynotes-project
```

Verify:

```
bash
```

```
gcloud iam service-accounts list
# Should show: github-actions-sa@askmynotes-project.iam.gserviceaccount.com
```

Step 9.2 — Grant Permissions to Service Account

```
bash

# Permission to deploy to Cloud Run
gcloud projects add-iam-policy-binding askmynotes-project \
  --member="serviceAccount:github-actions-sa@askmynotes-project.iam.gserviceaccount." \
  --role="roles/run.admin"

# Permission to push Docker images to Artifact Registry
gcloud projects add-iam-policy-binding askmynotes-project \
  --member="serviceAccount:github-actions-sa@askmynotes-project.iam.gserviceaccount." \
  --role="roles/artifactregistry.writer"

# Permission to submit Cloud Builds
gcloud projects add-iam-policy-binding askmynotes-project \
  --member="serviceAccount:github-actions-sa@askmynotes-project.iam.gserviceaccount." \
  --role="roles/cloudbuild.builds.editor"

# Permission to act as the service account (required for WIF)
gcloud iam service-accounts add-iam-policy-binding \
  github-actions-sa@askmynotes-project.iam.gserviceaccount.com \
  --member="serviceAccount:github-actions-sa@askmynotes-project.iam.gserviceaccount." \
  --role="roles/iam.serviceAccountTokenCreator"
```

Step 9.3 — Set Up Workload Identity Federation (WIF)

WIF lets GitHub Actions authenticate with GCP **without any key files**. This is the secure, modern approach — no JSON keys that can leak.

```
bash

# Step 1: Create the WIF pool
gcloud iam workload-identity-pools create "github-pool" \
  --location="global" \
  --display-name="GitHub Actions Pool" \
  --project=askmynotes-project
```

```
bash
```

```
# Step 2: Create the OIDC provider inside the pool
gcloud iam workload-identity-pools providers create-oidc "github-provider" \
  --location="global" \
  --workload-identity-pool="github-pool" \
  --display-name="GitHub OIDC Provider" \
  --issuer-uri="https://token.actions.githubusercontent.com" \
  --attribute-mapping="google.subject=assertion.sub,attribute.repository=assertion.r
  --attribute-condition="assertion.repository=='YOUR_GITHUB_USERNAME/YOUR_REPO_NAME'
  --project=askmynotes-project
```

Replace `YOUR_GITHUB_USERNAME/YOUR_REPO_NAME` with your actual repo, e.g.

`satyam/askmynotes`

bash

```
# Step 3: Get your GCP project number (different from project ID)
gcloud projects describe askmynotes-project --format="value(projectNumber)"
# Output: 123456789012
# Save this number – you'll need it below
```

bash

```
# Step 4: Allow GitHub Actions to impersonate the service account
# Replace PROJECT_NUMBER with the number from step 3
# Replace YOUR_GITHUB_USERNAME/YOUR_REPO_NAME with your repo
gcloud iam service-accounts add-iam-policy-binding \
  github-actions-sa@askmynotes-project.iam.gserviceaccount.com \
  --member="principalSet://iam.googleapis.com/projects/PROJECT_NUMBER/locations/glob
  --role="roles/iam.workloadIdentityUser" \
  --project=askmynotes-project
```

bash

```
# Step 5: Get the WIF provider resource name (you'll add this to GitHub secrets)
gcloud iam workload-identity-pools providers describe github-provider \
  --location="global" \
  --workload-identity-pool="github-pool" \
  --project=askmynotes-project \
  --format="value(name)"
# Output: projects/123456789012/locations/global/workloadIdentityPools/github-pool/p
# Save this entire string
```

Step 9.4 — Add Secrets to GitHub Repository

1. Go to your GitHub repo
2. Click **Settings** → **Secrets and variables** → **Actions**

3. Click **"New repository secret"** and add these one by one:

Secret Name	Value
GCP_PROJECT_ID	askmynotes-project
GCP_WIF_PROVIDER	The full provider name from Step 5 above
GCP_SERVICE_ACCOUNT	github-actions-sa@askmynotes-project.iam.gserviceaccount.com

Step 9.5 — Create GitHub Actions Workflow File

Create this file in your project: `.github/workflows/deploy.yml`

yaml

```
name: Build and Deploy to Cloud Run

on:
  push:
    branches:
      - main

env:
  PROJECT_ID: askmynotes-project
  REGION: us-central1
  REPO: askmynotes-repo
  SERVICE: askmynotes
  IMAGE: us-central1-docker.pkg.dev/askmynotes-project/askmynotes-repo/askmynotes

jobs:
  deploy:
    name: Build and Deploy
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write      # Required for WIF authentication

    steps:
      # 1. Checkout your code
      - name: Checkout code
        uses: actions/checkout@v4

      # 2. Authenticate to GCP using WIF (no key files)
      - name: Authenticate to GCP
        uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${{ secrets.GCP_WIF_PROVIDER }}
          service_account: ${{ secrets.GCP_SERVICE_ACCOUNT }}

      # 3. Set up gcloud CLI
      - name: Set up Cloud SDK
        uses: google-github-actions/setup-gcloud@v2
        with:
          project_id: ${{ secrets.GCP_PROJECT_ID }}

      # 4. Configure Docker to push to Artifact Registry
      - name: Configure Docker
        run: gcloud auth configure-docker us-central1-docker.pkg.dev --quiet

      # 5. Build Docker image using Cloud Build (on GCP servers)
      - name: Build image
```

```

run: |
    gcloud builds submit \
        --tag ${ env.IMAGE }:${ github.sha } \
        --timeout=15m \
        --quiet

# 6. Also tag as latest
- name: Tag as latest
  run: |
    gcloud artifacts docker tags add \
        ${ env.IMAGE }:${ github.sha } \
        ${ env.IMAGE }:latest

# 7. Deploy to Cloud Run
- name: Deploy to Cloud Run
  run: |
    gcloud run deploy ${ env.SERVICE } \
        --image ${ env.IMAGE }:${ github.sha } \
        --platform managed \
        --region ${ env.REGION } \
        --allow-unauthenticated \
        --memory 512Mi \
        --cpu 1 \
        --min-instances 0 \
        --max-instances 2 \
        --port 8080 \
        --timeout 120 \
        --quiet

# 8. Verify deployment succeeded
- name: Verify health check
  run: |
    URL=$(gcloud run services describe ${ env.SERVICE } \
        --region ${ env.REGION } \
        --format="value(status.url)")
    echo "App URL: $URL"

    # Wait 30 seconds for cold start
    sleep 30

    # Health check
    HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" "$URL/health")
    if [ "$HTTP_CODE" != "200" ]; then
        echo "Health check failed with HTTP $HTTP_CODE"
        exit 1
    fi
    echo "Health check passed – app is live at $URL"

```

```
# 9. Print the live URL in the Actions log
- name: Print deployment URL
  run: |
    URL=$(gcloud run services describe ${ env.SERVICE } \
      --region ${ env.REGION } \
      --format="value(status.url)")
    echo "✅ Deployed successfully!"
    echo "🔗 Live URL: $URL"
```

Push this file to trigger your first automated deploy:

```
bash

git add .github/workflows/deploy.yml
git commit -m "Add GCP Cloud Run CI/CD pipeline"
git push origin main
```

Go to your GitHub repo → **Actions** tab to watch the workflow run live.

PART 10 — Updating Your App

After the first setup, updating is simple:

```
bash

# Make your code changes, then:
git add .
git commit -m "Your change description"
git push origin main

# GitHub Actions automatically:
# 1. Builds new Docker image
# 2. Pushes to Artifact Registry
# 3. Deploys to Cloud Run
# 4. Runs health check
# Done in ~4 minutes
```

PART 11 — Useful Commands for Day-to-Day

```
bash
```

```
# View your live app URL
gcloud run services describe askmynotes \
  --region us-central1 \
  --format="value(status.url)"

# View recent logs (last 50 lines)
gcloud logging read \
  "resource.type=cloud_run_revision AND resource.labels.service_name=askmynotes" \
  --limit 50 \
  --format "table(timestamp, textPayload)"

# Stream live logs (like tail -f)
gcloud beta logging tail \
  "resource.type=cloud_run_revision AND resource.labels.service_name=askmynotes"

# View all deployed revisions
gcloud run revisions list --service askmynotes --region us-central1

# Roll back to previous revision if something breaks
gcloud run services update-traffic askmynotes \
  --region us-central1 \
  --to-revisions askmynotes-00001-xxx=100

# Update a single environment variable without redeploying
gcloud run services update askmynotes \
  --region us-central1 \
  --update-env-vars "GROQ_API_KEY=new-key-here"

# Check current env vars (values are hidden for security)
gcloud run services describe askmynotes \
  --region us-central1 \
  --format="yaml(spec.template.spec.containers[0].env)"

# Delete service if you want to stop it
gcloud run services delete askmynotes --region us-central1
```

Troubleshooting — Common Errors

Error: "PERMISSION_DENIED: iam.serviceaccounts.actAs"

```
bash
```

```
gcloud projects add-iam-policy-binding askmynotes-project \
  --member="serviceAccount:github-actions-sa@askmynotes-project.iam.gserviceaccount."
  --role="roles/iam.serviceAccountUser"
```

Error: "The user does not have permission to access the billing account"

You need to link billing manually:

1. Go to <https://console.cloud.google.com/billing>
2. Link `askmynotes-project` to your billing account

Error: "Container failed to start — port 8080 was not listening"

Your FastAPI app must listen on port 8080. Check your Dockerfile CMD:

dockerfile

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

And NOT `--port 8000`.

Error: "Cloud Build timed out"

Increase timeout:

bash

```
gcloud builds submit \
  --tag us-central1-docker.pkg.dev/askmynotes-project/askmynotes-repo/askmynotes \
  --timeout=20m
```

Error: "gcloud: command not found" after install

bash

```
# Add to PATH manually
export PATH="$HOME/google-cloud-sdk/bin:$PATH"
# Make permanent — add to ~/.bashrc or ~/.zshrc
echo 'export PATH="$HOME/google-cloud-sdk/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Cold Start Issue — App takes 30s to respond first time

This is normal with `--min-instances 0`. The first request after a period of no traffic takes 20-30s.

Options:

- Accept it (it's free, just demo by visiting the URL once before your interview)
- Set `--min-instances 1` (costs ~\$3-5/month — not recommended)

Health check shows Qdrant not connected

Your env vars may not have saved correctly:

```
bash

# Re-set all env vars
gcloud run services update askmynotes \
  --region us-central1 \
  --set-env-vars "QDRANT_URL=https://your-url:6333,QDRANT_API_KEY=your-key,GROQ_API_
```

Free Tier Limits — What to Watch

Resource	Free Tier Limit	Your Usage	Safe?
Cloud Run requests	2M / month	Student project: ~500	✅ Way under
Cloud Run CPU	180,000 vCPU-seconds / month	~1000 seconds	✅ Way under
Cloud Run RAM	360,000 GB-seconds / month	~500 GB-seconds	✅ Way under
Artifact Registry	0.5 GB free	1 image ~200MB	✅ Under
Cloud Build	120 build-minutes / day	5-8 min per deploy	✅ Under

Bottom line: A student portfolio project will never exceed these limits. The billing alert at \$0.01 is just insurance.

Summary Checklist

One-time setup:

- ☐ Created GCP account (credit card verified but not charged)
- ☐ Installed gcloud CLI
- ☐ Ran `gcloud auth login`
- ☐ Created project `askmynotes-project`
- ☐ Linked billing account to project
- ☐ Enabled 3 APIs (run, artifactregistry, cloudbuild)
- ☐ Created Artifact Registry repo
- ☐ Ran `gcloud auth configure-docker`
- ☐ Set \$0.01 billing alert

First deployment:

- ☐ Dockerfile exists with port 8080
- ☐ .dockerignore excludes .env
- ☐ Tested Docker locally
- ☐ Built and pushed image with `gcloud builds submit`
- ☐ Deployed with `gcloud run deploy`
- ☐ Set all env vars with `gcloud run services update --set-env-vars`
- ☐ Tested live URL `/health` endpoint

CI/CD setup:

- ☐ Created service account `github-actions-sa`
- ☐ Granted 3 IAM roles
- ☐ Created WIF pool and provider
- ☐ Added 3 secrets to GitHub repo
- ☐ Created `.github/workflows/deploy.yml`
- ☐ Pushed to main — watched Actions run successfully

After all this: Every future deploy = `git push origin main`. That's it.